JJ COLLEGE
ENGINEERING AND TECHNOLOGY
AUTONOMOUS
SOWDAMBIKAA GROUP OF INSTITUTIONS
ESTD. 1994

AI-PDLC HACKATHON · STUDENT KICKOFF

Build it like a product team.

From a real problem to a working prototype in 1.5 days, with AI at every step.

24 – 25 October 2026

WHY THIS ONE IS DIFFERENT

You won’t just write code. You’ll ship a product.

01

Plan before you build

No team writes code until its backlog passes a mentor’s check.

02

AI on every step

Use AI to research, write, design, code and test. Then question what it gives you.

03

Every feature traces back

Demo → story → epic → PRD feature → user need. A broken chain costs marks.

THE JOURNEY

Six stages, each with a gate to pass

1 Frame the problem

Make: problem pick + 5-line scope note

Gate: can you demo it in 1.5 days?

2 MRD

Make: users, personas, need, alternatives (2–3 pages)

Gate: clear user, real need, assumptions marked

3 PRD

Make: features ranked Must / Should / Could / Won't, metrics, out of scope

Gate: 3 Must-Haves at most, measurable metrics

4 Epics & stories

Make: 3–5 epics, 8–15 stories with acceptance criteria

Gate: small, testable stories that trace to the PRD

5 Build

Make: working prototype of the Must-Have stories

Gate: it runs, and every feature maps to a story

6 Demo

Make: live demo + slides from the team template

Gate: scored by judges on the rubric

YOUR TEAM OF FOUR

One point of contact for each area

BA

Business Analyst

Owns: MRD, PRD, epics and stories

Ask them: "Should we build this?"

UI/UX

Designer

Owns: user flows, screens, demo deck

Ask them: "How should this look and feel?"

Dev

Developer

Owns: repo, stack, the running build

Ask them: "Can we build this in time?"

QA

Quality Analyst

Owns: acceptance criteria, testing, traceability

Ask them: "Is it actually done?"

A point of contact keeps the focus and answers questions. **The work itself rotates:** everyone prompts, everyone reviews.

WORKING IN PARALLEL

Nobody waits for their turn

LEADS = owns this stage

	1 Frame	2 MRD	3 PRD	4 Backlog	5 Build	6 Demo
BA	**LEADS** picks the problem, writes the scope note	**LEADS** personas, need, alternatives	**LEADS** features, priorities, metrics	**LEADS** epics and user stories	Answers scope questions, cuts features	Tells the story on stage
UI/UX	Pictures who the user is	Studies how rival apps look and feel	Sketches flows for each Must-Have	Wireframes every Must-Have screen	Builds screens with a taste skill	**LEADS** demo deck and storyboard
Dev	Checks what's buildable in 1.5 days	Scouts GitHub for starters, APIs, tools	Picks the stack, flags risky features	Scaffolds the repo, sizes stories	**LEADS** builds story by story	Drives the live demo
QA	Lists what could go wrong	Fact-checks AI claims, marks assumptions	Makes every metric testable	**LEADS** acceptance criteria	Tests each story as it lands	Runs the traceability check

USING AI WELL

Six habits of teams that get the most out of AI

1 Let AI write your prompts

Try: “Write a prompt that gets a strong MRD for [problem]. Ask me 5 questions first.”

2 Make it interview you

Run grill-me before any document. It asks one sharp question at a time.

3 Ask for critique, not praise

“Review this PRD as a strict examiner. List the 5 weakest points.”

4 One story per prompt

Paste the story and its acceptance criteria as the spec. Never “build the whole app.”

5 Paste the exact error

Error, file and what you expected. Ask for the smallest fix, not a rewrite.

6 Keep a team brief

Your MRD summary, stack and rules in one file. In Claude Code, save it as CLAUDE.md.

SKILLS TOOLKIT

The right skill for each role

Install any skill: `npx skills add <repo> --skill <skill-name>`

Role	Skill	Repo	What it gives you
BA	grill-me	mattpocock/skills	Interviews you until the idea holds up
UI/UX	design-taste-frontend	Leonxlnx/taste-skill	Screens that don't look AI-generated
UI/UX	frontend-design	anthropics/skills	A bold, deliberate visual direction
Dev	brainstorming	obra/superpowers	Turns a rough idea into a plan first
Dev	test-driven-development	obra/superpowers	Test first, then the code
QA	webapp-testing	anthropics/skills	Tests your app in a real browser
QA	systematic-debugging	obra/superpowers	Finds the root cause before the fix

Skills load into Claude Code. Install them in your project folder and try each one before Day 1.

WHAT YOU HAND IN

Eight things, one submission

MRD
Users, need, alternatives, assumptions marked

BA

PRD
Must-Haves, metrics, what you won't build

BA

Backlog
Epics, stories and acceptance criteria

BA + QA

Repo + README
A stranger can run it in under 5 minutes

Dev

Prototype
Must-Have stories running live, plus a backup screen recording

Dev

Demo deck
Six slides from the team template

UI/UX

AI log
3–5 prompts that mattered, and what the AI got wrong

Everyone

Test checklist
Each acceptance criterion marked pass or fail

QA

COMMON TRAPS

Six ways good teams lose marks

⚠ “Build the whole app” in one prompt

You get code that won’t run. Go story by story.

⚠ Trusting AI’s statistics

AI invents numbers. Mark any figure you can’t verify as an assumption.

⚠ Polishing before it works

Pass the acceptance criteria first. Styling comes last.

⚠ Everything is a Must-Have

Cap it at three. Move the rest to Should or Won’t.

⚠ Regenerating to fix one bug

You lose working code. Paste the exact error and ask for the smallest fix.

⚠ Demoing features you never planned

If it’s not in your backlog, it breaks traceability and costs marks.

RUN OF SHOW

Two days, your pace

DAY 1

24 October

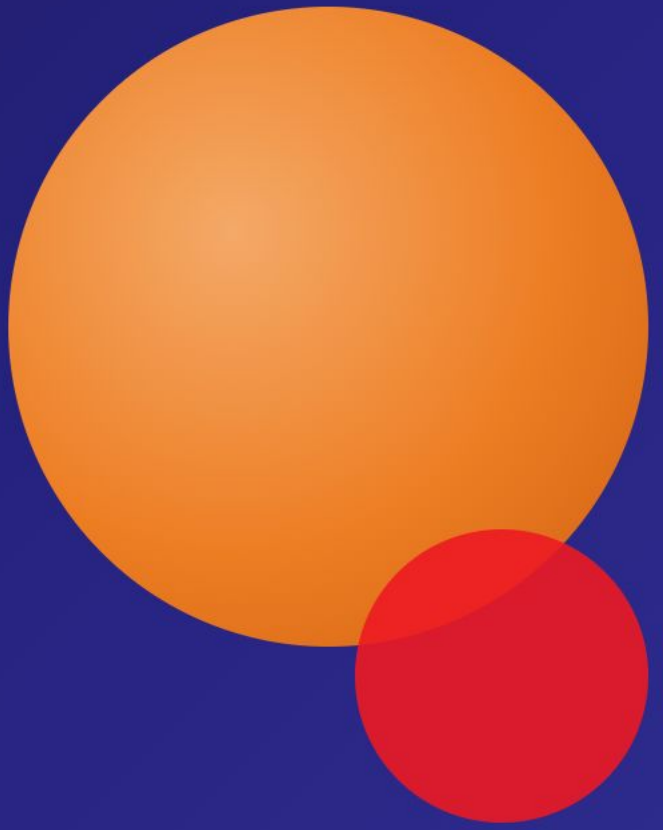
09:00–09:30	Kickoff presentation
09:30 on	Hackathon. Plan and build at your own pace. Keep going past 5 PM if you want.

DAY 2

25 October

10:00–11:30	Demos in 5 parallel tracks, 5 teams per track, 15 minutes per team
12:00	Closing. Results announced later.

Questions?



JJ COLLEGE
ENGINEERING AND TECHNOLOGY
AUTONOMOUS
SOWDAMBIKAA GROUP OF INSTITUTIONS
ESTD. 1994